

— INTERNAL ENABLEMENT

Agents that are not pods. kagent on Agent Substrate.

Four agents, two shared workers, no pod per agent. This deck covers how the pieces fit, which project owns which piece, and how to stand the whole thing up and demo it.

Audience: SA / FDE

substrate 0.0.9 · kagent 0.10.1

Verified 2026-09-21

— WHY THIS EXISTS

Idle agents cost you capacity, not CPU.

● AN AGENT AS AN ALWAYS-ON POD

- ◆ Reserves CPU and memory forever, whether or not anyone is talking to it.
- ◆ Uses close to nothing while idle, so the reservation is the real cost.
- ◆ Density is capped by what the scheduler has promised, not by what runs.
- ◆ Every agent you add is another permanent tenant of the cluster.

● AN AGENT AS A SNAPSHOT-BACKED ACTOR

- An idle agent is a compressed memory image in object storage.
- Workers are pre-warmed and shared. An agent occupies one only while it runs.
- Restore, serve the turn, checkpoint back. Sub-second on a warm pool.
- The sandbox is gVisor, so agent code runs against a user-space kernel.

Reservation is the honest comparison. Say that out loud in a demo, because it is the claim a skeptical platform engineer will test, and it survives testing.

— THE NUMBER TO REMEMBER

4 agents 0 pods

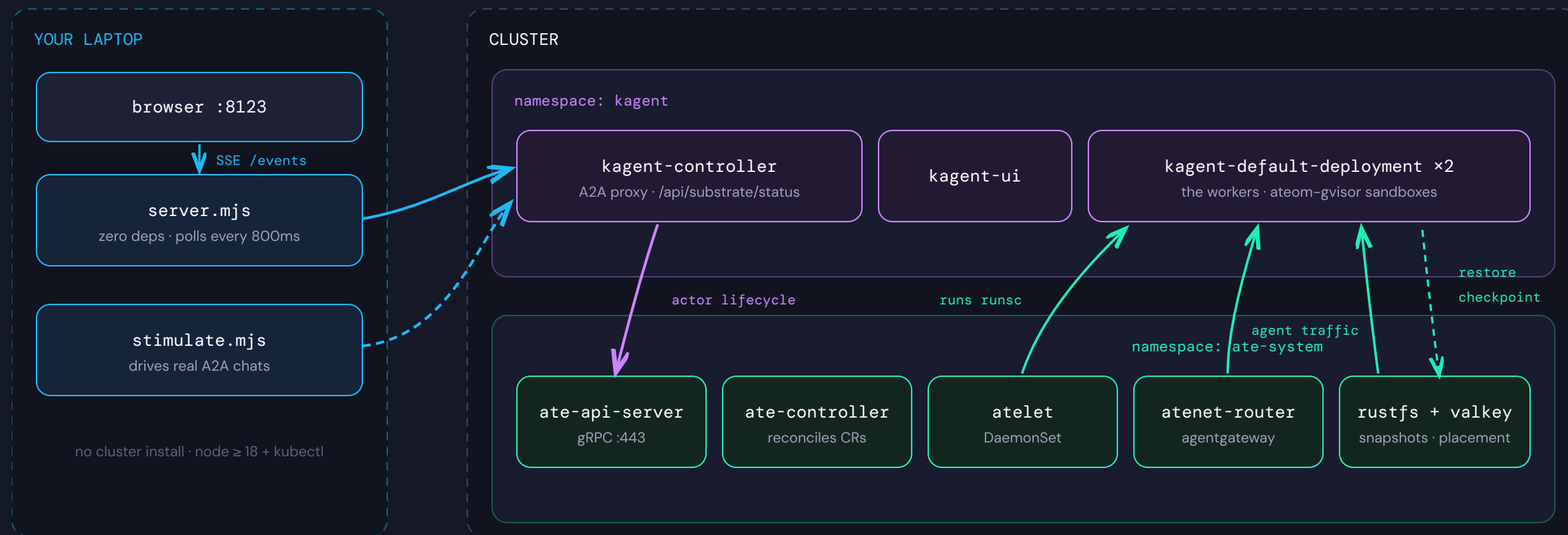
Four SandboxAgents, zero pods of their own. They share two pre-warmed workers.

- ◆ `kubectl get pods` shows the workers, never the agents. That is the demo in one command.
- ◆ Each agent has a golden snapshot in object storage and a placement record in the state store.
- ◆ Adding a fifth agent adds a snapshot, not a pod.
- ◆ The pool scales on queue depth and busy slots, not on CPU.

— ARCHITECTURE

Three tiers. Only two of them run in the cluster.

■ Agent Substrate ■ kagent ■ Substrate Scope



Scope reaches the cluster with kubectl and two long-lived port-forwards. stimulate.mjs posts A2A message/send straight at the controller.

— OWNERSHIP

Three separate projects. Know which one you are debugging.

kagent ships the integration, not the runtime. Substrate installs separately, and Scope is not installed at all.

■ AGENT SUBSTRATE

The runtime

Runs actors in gVisor sandboxes, snapshots them, restores them, routes to them. Installs into **ate-system** from its own charts. Knows nothing about agents.

Owns: WorkerPool, ActorTemplate, SandboxConfig

■ KAGENT

The agent platform

Turns a declarative agent into an ActorTemplate, proxies A2A traffic to it, and exposes substrate state over REST. Installs into **kagent**. Knows nothing about gVisor.

Owns: SandboxAgent, the controller, the UI

■ SUBSTRATE SCOPE

The view

A local Node process that reads the cluster and draws it. No operator, no CRD, no pod. Apache-2.0, maintained outside Solo.

Owns: nothing in the cluster

Substrate is published by the kagent org at ghcr.io/kagent-dev/substrate, which makes it look bundled. It is not. You install two charts, then kagent.

Eight components, and only one of them touches the sandbox.

ate-api-server

The control plane. gRPC on :443. Creates, suspends, resumes and deletes actors. Validates callers with service-account tokens.

ate-controller

Reconciles the ate.dev CRs. Turns an ActorTemplate into a baked golden snapshot.

atelet

DaemonSet, one per node. Downloads the gVisor release, extracts runsc, starts and supervises sandboxes, drives snapshots. **This is the only component that runs runsc.**

atenet-router

The actor data path. Runs agentgateway and routes by Host header to <actor-id>.actors.resources.substrate.ate.dev.

rustfs

In-cluster S3-compatible object store. Every golden and incremental snapshot lands here.

valkey, 6 nodes

The actor and worker state store. Which actor sits on which worker is a record here, not a Kubernetes object.

Also present: atenet-dns for actor name resolution, and a pod-certificate controller that needs three alpha feature gates. Those gates are why this does not run on a stock cluster.

kagent contributes four things, and one of them is the workers.

The SandboxAgent CRD

An ordinary declarative kagent agent with one extra block. **spec.substrate is the only substrate-specific thing in the manifest.** The same agent definition runs as an always-on pod without it.

The controller

Reconciles each SandboxAgent into a generated ActorTemplate, calls ate-api for lifecycle, and proxies A2A requests at `/api/a2a-sandboxes/<ns>/<name>/.`

The inventory API

GET `/api/substrate/status` aggregates WorkerPools, ActorTemplates and live actor and worker state from ate-api into one REST payload. Scope is built on it.

The WorkerPool

The kagent chart creates the pool when `substrateWorkerPool.create` is true, and its pods are the pre-warmed sandboxes. Two replicas, because agent rollouts are blue-green.

The controller also runs the ordinary pod-based agents, k8s-agent and the rest. Having both kinds on one cluster is useful: it is the control when something breaks.

One Node process, zero dependencies, nothing installed.

server.mjs

HTTP server on :8123. Polls the cluster every 800ms and pushes state to the browser over SSE at /events.

public/index.html

The whole UI in one file. No build step, no framework, no bundler.

stimulate.mjs

A separate process. Drives real A2A chats and reports each one back to server.mjs over HTTP.

● HOW IT REACHES THE CLUSTER

- Shells out to **kubectl**, every call pinned with the context you gave it.
- Spawns two long-lived port-forwards, to the controller and the UI. Both respawn on exit.

● WHERE IT WRITES, NOT JUST READS

- ◆ Worker plus and minus, and the autoscaler, run **kubectl scale workerpool**.
- ◆ RESET POOL runs a rollout restart of the worker Deployment.

— OBJECT MODEL

One agent you author becomes four objects you can inspect.



You write the first one

A SandboxAgent is a declarative agent plus **spec.substrate.workerPoolRef**.
The kagent controller generates everything downstream.

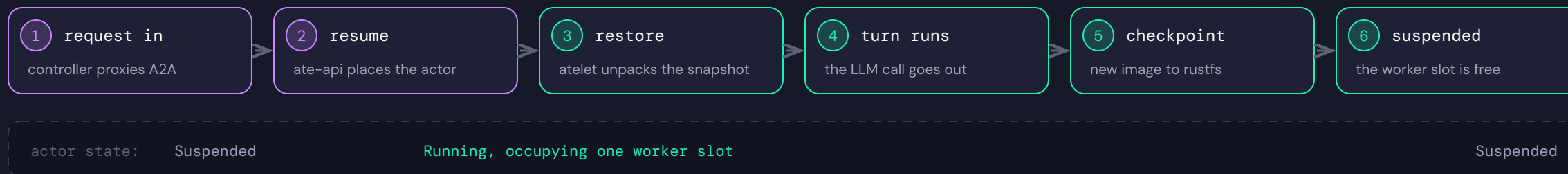
The last one is invisible to kubectl

An Actor is a record in valkey reached through ate-api, not a Kubernetes resource. That single fact explains most of what follows.

Baking a golden means substrate started the agent container inside runsc on a worker, let it reach ready, then checkpointed the whole process image. A Ready SandboxAgent is proof gVisor works.

— ONE TURN, END TO END

The agent exists only while it is answering.



Why the pool wants two workers

A config or image change to a SandboxAgent is blue-green. kagent keeps the old ActorTemplate serving until the new golden is Ready, so a single-worker pool has **nowhere to bake the replacement**.

What happens when the pool is full

Substrate **rejects rather than queues**. The restore queue you see on the board is drawn from client-side retry reports, not from a queue inside substrate. That is worth saying if someone asks where the queue lives.

The first turn after an install is the slowest you will see, because the actor is cold. Send one throwaway chat before a demo so you do not open on the worst number.

— WHERE THE STATE LIVES

kubectl cannot show you the interesting half.

Snapshots live in rustfs and placement records live in valkey, reachable only through ate-api. That is why Scope has two adapters with very different fidelity.

■ source: kagent

Everything

Reads **/api/substrate/status** on the controller, which proxies ate-api, plus the sessions API for chats. Per-actor state, worker occupancy, the drawer, SURGE.

Needs kagent with substrate enabled

■ source: crd

The Kubernetes-visible half

Reads WorkerPools, worker pods and ActorTemplates with kubectl. No actor placement at all, so it synthesizes one pseudo-actor per template.

Works on any substrate cluster, no kagent

Scope picks the source automatically. On our bundle it picks kagent. That REST endpoint is also the one kagent 1.0.0-alpha1 removed, which is why the bundle pins 0.10.1.

— WORTH KNOWING

agentgateway is already the actor data path.

In substrate today

atenet-router runs agentgateway with networking-mode set to agentgateway. Every request that reaches an actor is routed by it, keyed on the actor Host header. This is true at the version the bundle pins, not a roadmap item.

In the newer substrate line

An atenet-egress component adds a second agentgateway doing HTTPS interception on actor egress, with credential injection from Kubernetes Secrets. An actor can then call an upstream API without ever holding the key.

Do not confuse this with putting a Solo-configured agentgateway in front of kagent for LLM routing, MCP tool RBAC or token exchange. That is a different job and a separate bundle.

— SET EXPECTATIONS

What bites, said out loud.

Agent Substrate is early. Demo it as a shape of an answer, not a product you would put in front of production traffic next quarter.

- ◆ **The version pairing is tight.** kagent 0.10.1 vendors substrate 0.0.9. The newest kagent removes the REST endpoint Scope depends on, so newer is worse for this demo.
- ◆ **In-place substrate upgrades are not safe** across patch versions. Actor records are stored as protojson and the schema moved. Rebuild the cluster instead.
- ◆ **Actor egress breaks on any overlay CNI.** The gVisor sandbox does not inherit the pod MTU, so outbound TLS blackholes on Calico, Cilium or flannel. We clamp MSS on the node to work around it.
- ◆ **There is no exec or SSH into an actor.** The only way in is network ingress through atenet, which is why interaction rides on A2A.
- ◆ **The cluster is not ordinary.** Three alpha feature gates and Kubernetes 1.36, or nothing starts.

Full evidence, with verbatim errors and the controlled comparison behind the MTU finding, is in the bundle at docs/findings.md.

— WHAT TO SHOW, AND WHAT IT PROVES

Three beats, twelve minutes.

01

Count the pods

Four agents defined. No pods running them. Then show the ActorTemplates and their snapshots.

02

Watch one turn

Chat an agent and follow the chip: storage, queue, worker, storage. Open the drawer for latency.

03

Drive it

Start the stimulator. The queue becomes visible, and reserved capacity separates from the always-on line.

04

Scale it

Worker plus and minus write to the real CR. Turn on the autoscaler and let demand drive it.

The question you will get

"Is this production ready?" No. Say so, then say what is real: snapshot-restore density plus gVisor isolation, with agentgateway already routing every request that reaches an actor.

— APPENDIX A

Stand it up in one command.

Cluster, platform and agents from nothing. Measured at 2 minutes 25 seconds with a warm image cache.

```
# cluster + substrate + kagent + 4 agents + all 6 tests
solomog vind:create apply test CLUSTER=kas BUNDLE=kagent-substrate \
  VCLUSTER_VALUES=bundles/kagent-substrate/helpers/vcluster-substrate.yaml

# iterate on one test by filename prefix
solomog test BUNDLE=kagent-substrate CLUSTER=kas TESTS=60

# tear it down
solomog teardown CLUSTER=kas
```

What the config file adds

Three alpha feature gates, a pod-reachable jwks_uri, and the MSS clamp as a systemd unit so it survives a restart. Without any one of them the install fails in a way that looks like something else.

What you need first

A model credential in .env matching KAGENT_PROVIDER, Docker Desktop with room for roughly 20 pods, and node 18 or newer if you want the board.

If a test fails

Read the capture under .solomog/test-runs. The tests are ordered cheap control first, so the first failure is usually the real one.

— APPENDIX B

Running the board.

```
# the board. scope is cloned on demand into .solomog/ and pinned
bash bundles/kagent-substrate/helpers/scope.sh kas
#   → http://localhost:8123   and the kagent UI on :8001

# drive real traffic at the agents, in a second terminal
bash bundles/kagent-substrate/helpers/scope.sh kas --stimulate

# no cluster needed, for rehearsing the narration
bash bundles/kagent-substrate/helpers/scope.sh kas --sim
```

Before anyone joins

Run the tests green, open the board, and send one throwaway chat so the first turn of the demo is not the cold one.

One thing to remember

kubect! scale takes field ownership of spec.replicas, so a later apply that manages the pool may need force-conflicts.

— APPENDIX C

Where to read more.

docs/findings.md

Why the pins are what they are, the jwks_uri trap, the MTU comparison, and every gotcha with its verbatim error.

docs/demo-runsheet.md

The beat-by-beat script, written to be read at the keyboard rather than projected.

docs/issue-ateom-gvisor-mtu.md

A drafted upstream issue for the MTU defect, with a pre-filing checklist. Not filed yet.

Pinned versions

substrate 0.0.9, kagent 0.10.1, Scope at commit 5d27549. Override each with an env var if you are testing a newer line.

The bundle

bundles/kagent-substrate. Root hooks install, tests verify, helpers hold the cluster config and the Scope runner.

Upstream

agent-substrate/substrate, kagent-dev/kagent, themSquared/substrate-scope. All Apache-2.0.